

RELAZIONE TECNICA

Esercizio Linux Shell, processi e ambiente di sviluppo su Kali Linux

Corso: Cyber Security & Ethical Hacking

Studente: D'Angelo Giovanni

1. Obiettivo dell'attività

Con questa attività ho messo in pratica i concetti fondamentali introdotti nel modulo Linux: uso della shell, gestione del file system, spostamento tra directory tramite percorsi relativi e assoluti, copia e spostamento di file, rimozione di cartelle, modifica dei permessi, osservazione dei processi in esecuzione e verifica dell'ambiente di sviluppo con Visual Studio Code e GDB.

2. Ambiente utilizzato

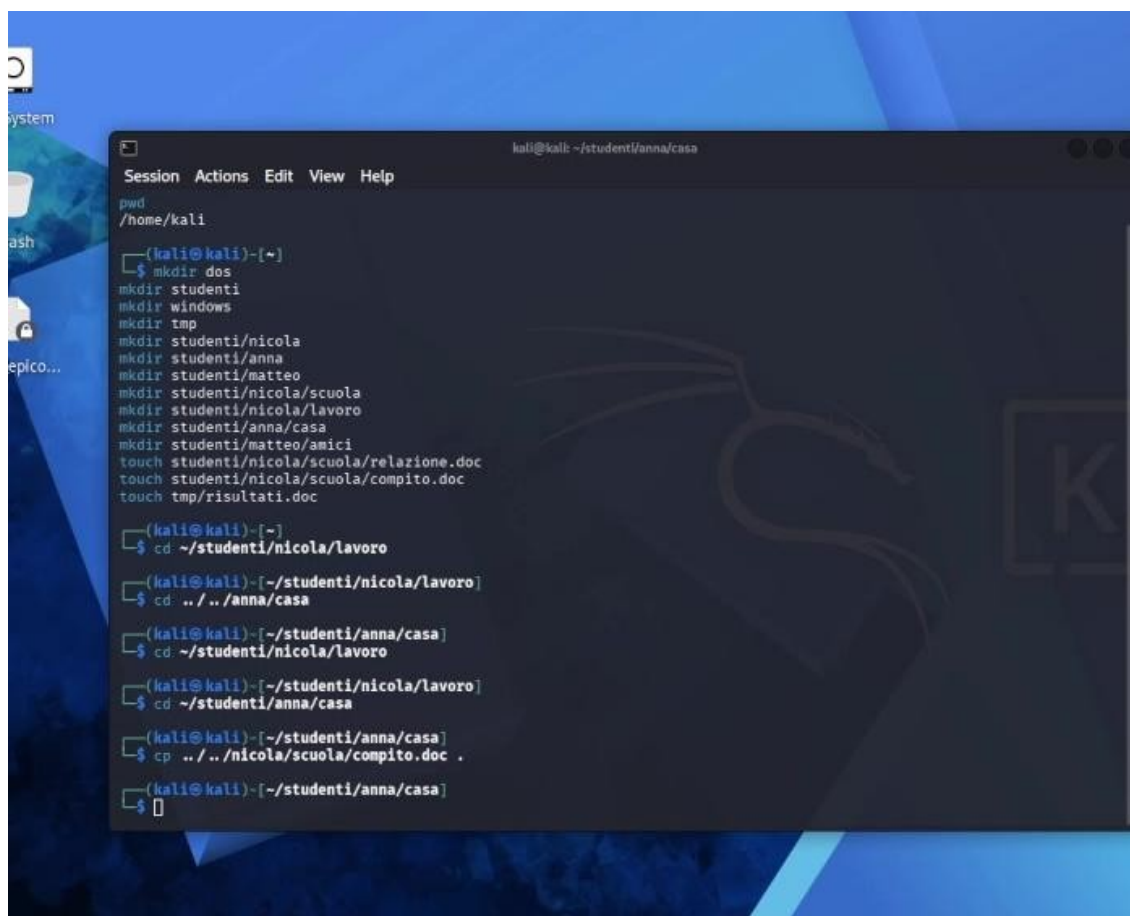
Ho svolto l'esercizio in ambiente Kali Linux, utilizzando principalmente il terminale per l'esecuzione dei comandi richiesti. Nella parte finale ho utilizzato anche il browser, la cartella Downloads e Visual Studio Code per verificare la disponibilità dell'ambiente di sviluppo richiesto per le attività successive.

3. Svolgimento operativo

3.1 Creazione della struttura di directory e dei file richiesti

Partendo dalla home dell'utente, ho costruito nel terminale la struttura di cartelle prevista dalla traccia. Ho creato le directory principali dos, studenti, windows e tmp; successivamente ho generato le sottocartelle studenti/nicola, studenti/anna e studenti/matteo, con le rispettive diramazioni scuola, lavoro, casa e amici. Una volta definita la struttura, ho creato anche i file relazione.doc, compito.doc e risultati.doc tramite il comando touch.

Questa prima fase mi ha permesso di applicare in modo pratico i comandi mkdir e touch, cioè due strumenti di base della shell Linux per la creazione di directory e file.



```
kali@kali: ~/studenti/anna/casa
Session Actions Edit View Help
pwd
/home/kali

(kali@kali)-[*]
$ mkdir dos
mkdir studenti
mkdir windows
mkdir tmp
mkdir studenti/nicola
mkdir studenti/anna
mkdir studenti/matteo
mkdir studenti/nicola/scuola
mkdir studenti/nicola/lavoro
mkdir studenti/anna/casa
mkdir studenti/matteo/amici
touch studenti/nicola/scuola/relazione.doc
touch studenti/nicola/scuola/compito.doc
touch tmp/risultati.doc

(kali@kali)-[*]
$ cd ~/studenti/nicola/lavoro

(kali@kali)-[~/studenti/nicola/lavoro]
$ cd ../../anna/casa

(kali@kali)-[~/studenti/anna/casa]
$ cd ~/studenti/nicola/lavoro

(kali@kali)-[~/studenti/nicola/lavoro]
$ cd ~/studenti/anna/casa

(kali@kali)-[~/studenti/anna/casa]
$ cp ../../nicola/scuola/compito.doc .

(kali@kali)-[~/studenti/anna/casa]
$
```

Figura 1 - Creazione della struttura di directory e primi spostamenti tra i percorsi.

3.2 Spostamenti tramite percorsi relativi e assoluti

Dopo la creazione della struttura, mi sono posizionato nella directory `~/studenti/nicola/lavoro` e sono passato alla cartella `casa` dell'utente `anna` sia tramite percorso relativo sia tramite percorso assoluto. L'uso combinato dei due approcci mi ha permesso di verificare la differenza tra uno spostamento basato sulla posizione corrente e uno basato sull'indirizzo completo della directory di destinazione.

3.3 Operazioni su file, cartelle e permessi

Dalla directory `casa` di `anna` ho copiato il file `compito.doc` prelevandolo dal percorso di `nicola/scuola`, mentre `relazione.doc` è stato spostato nella stessa posizione tramite `mv`. In seguito ho eliminato la cartella `tmp` con `rm -r`, ho creato il file `pippo.txt` nella directory `lavoro` di `nicola` e ne ho modificato il permesso con `chmod 644`.

Sempre nella stessa fase ho rinominato la cartella `casa` in `.casa`, trasformandola in directory nascosta e mostrando in modo pratico l'effetto del punto iniziale nei nomi dei file e delle cartelle in Linux. Mi sono poi spostato nuovamente nella directory `lavoro` per visualizzare il contenuto di `pippo.txt`, ho rimosso la cartella `amici` e ho concluso con la pulizia finale delle cartelle create per l'esercitazione.

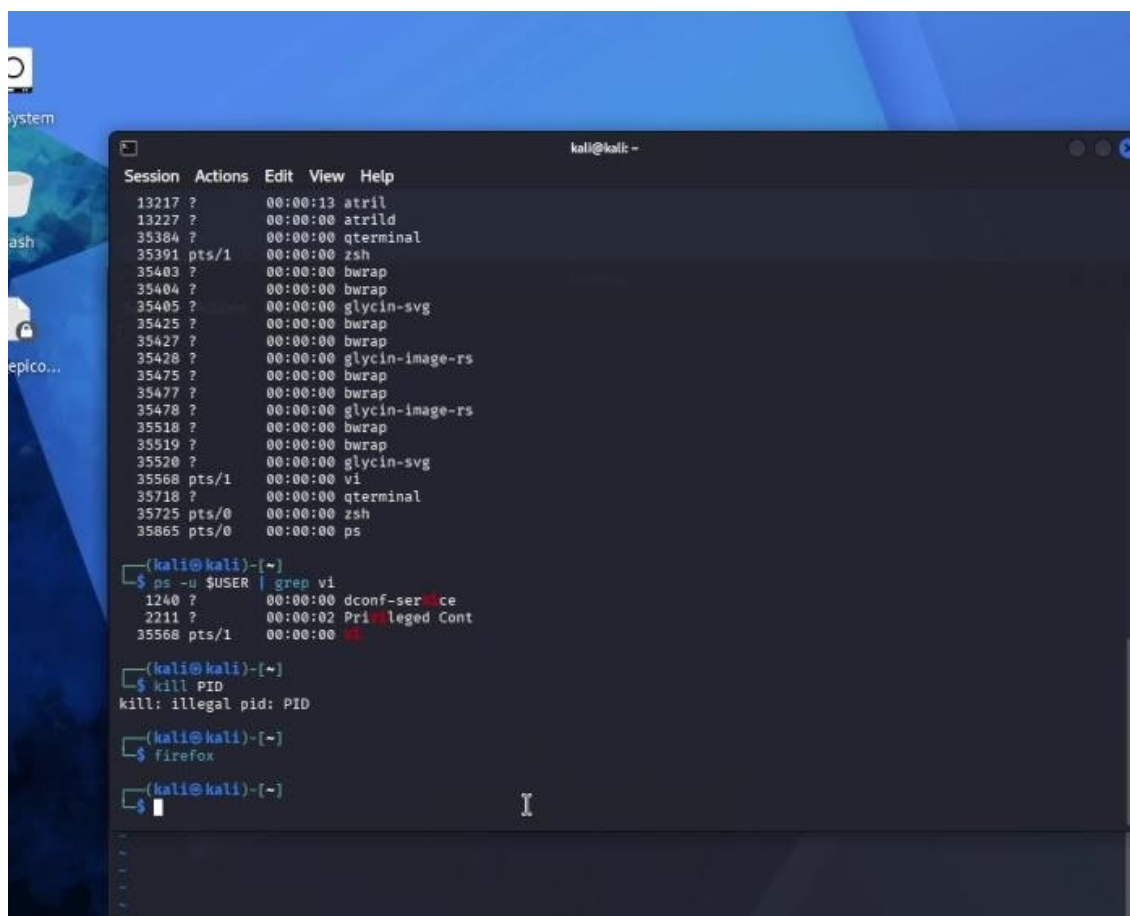


Figura 2 - Operazioni su file e cartelle: mv, rm -r, touch, chmod 644 e rinomina della directory casa in .casa.

3.4 Consultazione dei comandi di sistema e osservazione dei processi

Terminata la parte sul file system, mi sono concentrato sui processi. In questa sezione ho richiamato i comandi di base previsti dall'esercizio, tra cui w, who, who am i e i manuali di riferimento per ps e kill. L'obiettivo non era soltanto eseguire dei comandi, ma capire come osservare il sistema, identificare i processi attivi e usare gli strumenti di help integrati della shell.

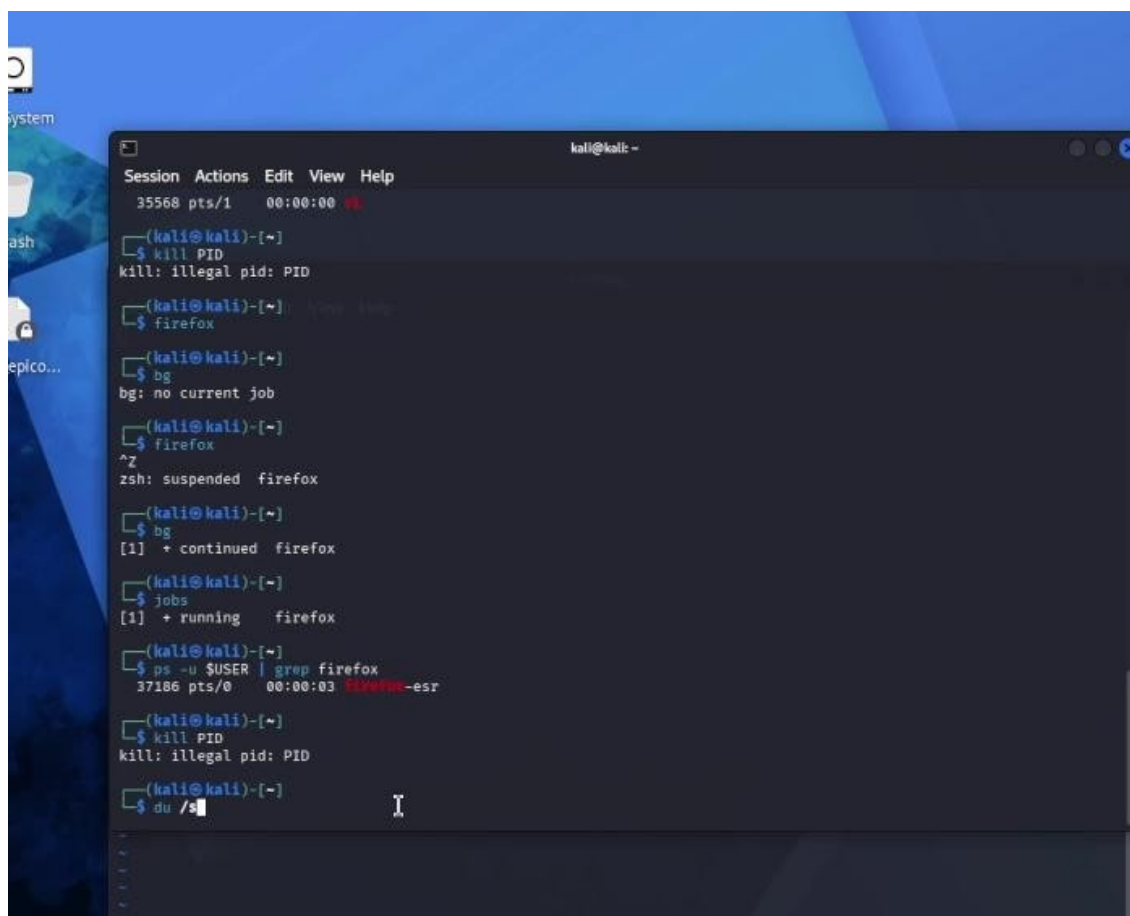
A screenshot of a Kali Linux terminal window. The window title is 'kali@kali: ~'. The terminal shows a series of commands and their outputs. At the top, it says 'Session Actions Edit View Help' and '35568 pts/1 00:00:00'. The commands and outputs are: 1. '\$ kill PID' followed by 'kill: illegal pid: PID'. 2. '\$ firefox' followed by a new prompt. 3. '\$ bg' followed by 'bg: no current job'. 4. '\$ firefox' followed by '^Z' and 'zsh: suspended firefox'. 5. '\$ bg' followed by '[1] + continued firefox'. 6. '\$ jobs' followed by '[1] + running firefox'. 7. '\$ ps -u \$USER | grep firefox' followed by '37186 pts/0 00:00:03 firefox-esr'. 8. '\$ kill PID' followed by 'kill: illegal pid: PID'. 9. '\$ du /s' followed by a cursor. The terminal has a dark background with light-colored text.

Figura 3 - Consultazione dei comandi di sistema e avvio della parte dedicata ai processi.

3.5 Apertura di vi, ricerca del processo e gestione di Firefox in background

Successivamente ho aperto l'editor vi su un file denominato pippo, quindi in un'altra sessione ho elencato i processi dell'utente con `ps -u $USER` e ho filtrato quelli contenenti vi tramite `grep`. Nel corso della prova ho digitato il comando `kill` con il segnaposto PID non ancora sostituito con l'identificativo numerico reale; il terminale ha quindi restituito il messaggio di errore relativo a un PID non valido. Questo passaggio, pur non portando all'interruzione del processo, mi ha mostrato in modo pratico che il comando `kill` richiede il numero reale del processo individuato da `ps`.

La prova è poi continuata con l'avvio di Firefox. Un primo tentativo di invio in background con `bg` è avvenuto quando non era ancora presente un job sospeso, e infatti la shell ha risposto con il messaggio `no current job`. Dopo la sospensione del browser con `Ctrl+Z`, ho eseguito nuovamente il comando `bg` con esito positivo, verificando poi la voce presente in `jobs` e filtrando il processo `firefox-esr` con `ps -u $USER | grep firefox`.

```
kali@kali: ~/Downloads
Session Actions Edit View Help
zsh: suspended firefox

(kali@kali)~$ bg
[1] + continued firefox

(kali@kali)~$ jobs
[1] + running firefox

(kali@kali)~$ ps -u $USER | grep firefox
37186 pts/0 00:00:03 firefox-esr

(kali@kali)~$ kill PID
kill: illegal pid: PID

(kali@kali)~$ du -sh ~
289M /home/kali

(kali@kali)~$ cd ~/Downloads
ls
csaag_3.128.0-1770000727_amd64.deb

(kali@kali)~/Downloads$ sudo apt update && sudo apt install gdb
[sudo] password for kali:
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [25.9 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:5 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.0 MB]
92% [5 Contents-amd64 46.4 MB/53.0 MB 88%]
1,246 kB/s 5s
```

Figura 4 - Ricerca del processo vi, utilizzo di kill e gestione di Firefox in background.

3.6 Verifica dello spazio occupato

Al termine della sezione dedicata ai processi ho lanciato il comando `du -sh ~` per controllare l'occupazione dello spazio nella home dell'utente. L'output del terminale ha restituito un valore sintetico riferito alla directory `/home/kali`, completando così anche l'ultima richiesta della parte facoltativa.

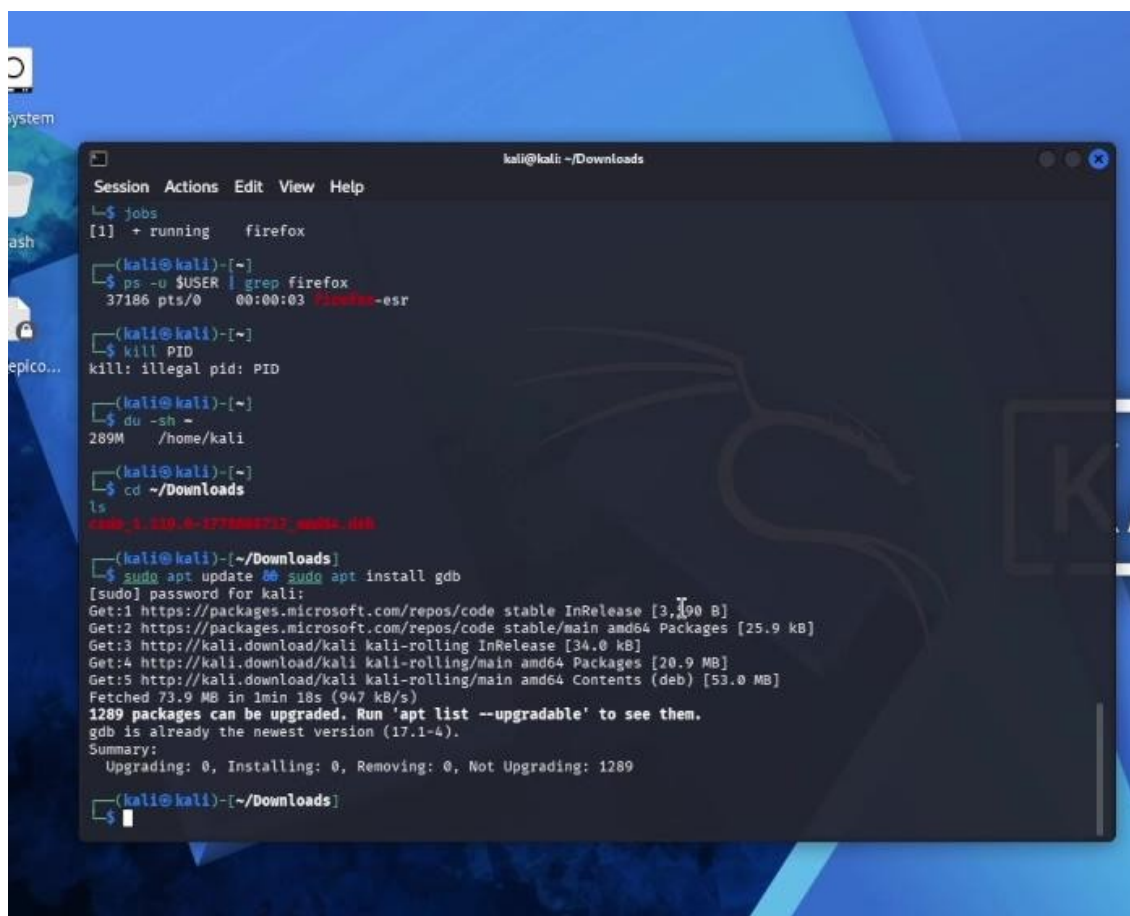
A screenshot of a Kali Linux terminal window. The window title is 'kali@kali: ~/Downloads'. The terminal shows the following commands and output:
- `jobs`: Shows '[1] + running firefox'.
- `ps -u $USER | grep firefox`: Shows '37186 pts/0 00:00:03 firefox-esr'.
- `kill PID`: Shows 'kill: illegal pid: PID'.
- `du -sh ~`: Shows '289M /home/kali'.
- `cd ~/Downloads`: Changes directory to ~/Downloads.
- `ls`: Shows 'code_1.210.0-1778000717_wndw.deb'.
- `sudo apt update && sudo apt install gdb`: Updates package lists and installs gdb. The output shows connections to Microsoft and Kali repositories, and a summary indicating 1289 packages can be upgraded.
- `gdb is already the newest version (17.1-4).`
- `Summary:`
 Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1289
- `(kali@kali)~/.Downloads`: Shows the current directory.
- `ls`: Shows a file icon.

Figura 5 - Verifica dello spazio occupato, accesso alla cartella Downloads e controllo del pacchetto .deb.

3.7 Verifica della presenza del pacchetto di Visual Studio Code e controllo di GDB

Nella parte finale ho aperto la cartella Downloads mediante `cd ~/Downloads` ed ho eseguito `ls`, così da controllare la presenza del pacchetto .deb di Visual Studio Code già disponibile nella cartella di download. Subito dopo ho lanciato il comando `sudo apt update && sudo apt install gdb` per aggiornare gli indici dei pacchetti e verificare la disponibilità del debugger GDB.

L'output del terminale ha mostrato il contatto con i repository di Kali e con il repository Microsoft relativo a Code, oltre al risultato finale secondo cui `gdb` risultava già presente come versione più recente. Questo ha confermato che il debugger era disponibile nel sistema e che l'ambiente era pronto per essere utilizzato nei moduli successivi.

3.8 Avvio e verifica dell'ambiente Visual Studio Code

Conclusa la parte da terminale, ho avviato Visual Studio Code dal menu applicazioni. La schermata iniziale del programma si è aperta correttamente, confermando il buon funzionamento dell'editor all'interno della macchina Kali Linux. In questo modo ho concluso l'esercizio con una verifica pratica dell'ambiente di lavoro richiesto per le attività future legate alla programmazione e al debug.

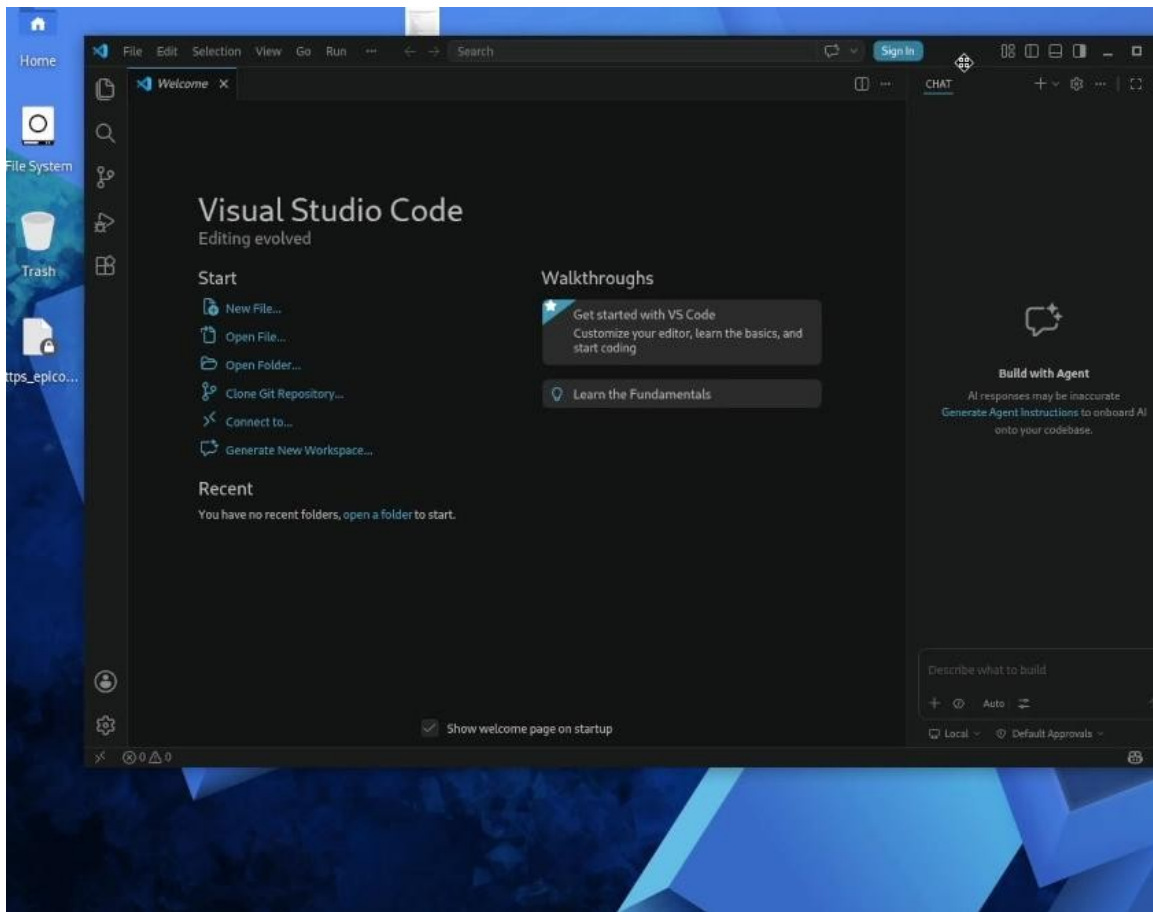


Figura 6 - Avvio di Visual Studio Code e verifica della corretta apertura dell'applicazione.

4. Risultati ottenuti

- Creazione corretta della gerarchia di directory e dei file previsti dalla traccia.
- Esecuzione di spostamenti tramite percorsi relativi e assoluti.
- Uso pratico dei comandi cp, mv, rm, chmod, ls e cd.
- Comprensione del significato operativo dei permessi attraverso l'applicazione di chmod 644 su pippo.txt.
- Osservazione dei processi di sistema con ps e grep e gestione di un'applicazione sospesa e ripresa in background con Ctrl+Z, bg e jobs.
- Verifica dello spazio disco con du -sh ~.
- Controllo della disponibilità dell'ambiente di sviluppo tramite presenza del pacchetto .deb, aggiornamento dei repository, verifica di GDB e apertura di Visual Studio Code.

5. Osservazioni tecniche

Durante la prova sono emersi due aspetti utili dal punto di vista didattico. Il primo riguarda il comando kill: usare la stringa letterale PID non produce l'effetto desiderato, perché il comando richiede il numero reale del processo. Il secondo riguarda bg: il comando funziona solo quando esiste un job sospeso; se non è presente alcun job, la shell restituisce correttamente il messaggio no current job. Entrambe le situazioni si sono rivelate utili per comprendere meglio il comportamento reale della shell.

6. Conclusione

L'esercitazione è risultata utile per consolidare l'uso della shell Linux in un contesto operativo concreto. Attraverso la creazione della struttura di cartelle, la gestione di file e permessi, l'osservazione dei processi e il controllo dell'ambiente di sviluppo, ho potuto mettere in pratica i concetti fondamentali del modulo Linux in modo coerente con gli obiettivi del corso.